Binary Classification of Pulmonary Nodules on Chest X-rays

EfficientNetV2–MEAM Architecture with SEPE Preprocessing

Clément B.

2026

Contents

1 Introduction

This document describes the architecture and operation of a binary classification pipeline for pulmonary nodules from chest X-rays. The system is based on three main components:

1. preprocessing with adaptive edge enhancement (**SEPE** — *Sparse Edge-Preserving Enhancement*),
2. a multi-scale attention mechanism (**MEAM** — *Multi-Scale Embedding Attention Mechanism*),
3. an **EfficientNetV2-S** backbone pre-trained on ImageNet, coupled with a linear classification head.

The dataset comes from the *NIH Chest X-ray Dataset*, filtered to keep only two classes: `Nodule` and `No Finding`.

2 Input construction: three-channel representation

Each grayscale chest X-ray is transformed into a **three-channel** tensor, designed to provide the network with complementary information of different types:

Channel	Name	Description
c_1	Raw density	Original image normalized with Min-Max in $[0, 1]$. Preserves radiological density information.
c_2	SEPE	Adaptive edge enhancement (Section 3). Computed on GPU during training via `_build_3ch_batch`.
c_3	CLAHE	*Contrast Limited Adaptive Histogram Equalization* (clipLimit = 2.0, grid 8×8). Improves local contrast in low-dynamic regions.

Table 1: Model input channels ($512 \times 512 \times 3$).

The `Dataset` loads only two channels on CPU (c_1 and c_3) to avoid a bottleneck. The SEPE channel (c_2) is computed on GPU just before inference thanks to the batch version of the preprocessor.

3 SEPE preprocessing

Sparse Edge-Preserving Enhancement enhances fine structures (nodule contours, bronchi) while limiting noise amplification. It is decomposed into three steps.

3.1 High-frequency enhancement

Let $I(x, y)$ be the input image and $G_\sigma = G_\sigma * I$ its smoothed version using a Gaussian kernel with standard deviation σ. The high-frequency detail $I - G_\sigma$ is weighted by an adaptive mask W:

$$I_{\text{enh}}(x, y) \;=\; I(x, y) \;+\; \alpha\, W(x, y)\, \big(I(x, y) - G_\sigma(x, y)\big) \tag{1}$$

where $\alpha = 1{,}5$ controls enhancement intensity.

3.2 Adaptive weighting mask

The mask W is built from the gradient magnitude in order to *preserve* existing edges (strong gradient $\Rightarrow W \to 0$) and *enhance* smooth areas (weak gradient $\Rightarrow W \to 1$):

$$W(x, y) \;=\; \exp\!\left(-\frac{\|\nabla I(x, y)\|^2}{\sigma_e^2}\right) \tag{2}$$

with $\sigma_e = 0{,}1$, which sets gradient sensitivity. The gradient is estimated with 3×3 Sobel filters.

3.3 Laplacian regularization

To prevent amplification of high-frequency noise, a regularization term based on the discrete Laplacian is subtracted:

$$I_{\text{SEPE}}(x, y) \;=\; I_{\text{enh}}(x, y) \;-\; \lambda\, \big|\nabla^2 I(x, y)\big| \tag{3}$$

with $\lambda = 0{,}1$. The result is then brought back into $[\min(I),\ \max(I)]$ by clipping.

3.4 GPU implementation (batch version)

During training, SEPE is computed on GPU through PyTorch convolutions (`F.conv2d`) to process a full batch in parallel. The operations exactly reproduce Equations (1)–(3) with:

- a separable 2D Gaussian kernel of size $\lceil 6\sigma \rceil | 1$,
- 3×3 Sobel filters for the gradient,
- the Laplacian kernel $\left[\begin{smallmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{smallmatrix}\right]$.

The SEPE channel is then normalized by dividing by the batch maximum to bring it into $[0, 1]$.

Parameter	**Value**	**Role**
σ	1,0	Standard deviation of Gaussian smoothing
α	1,5	Detail amplification factor
σ_e	0,1	Weight mask sensitivity
λ	0,1	Laplacian regularization coefficient

Table 2: SEPE preprocessor hyperparameters.

4 Model architecture

4.1 Overview

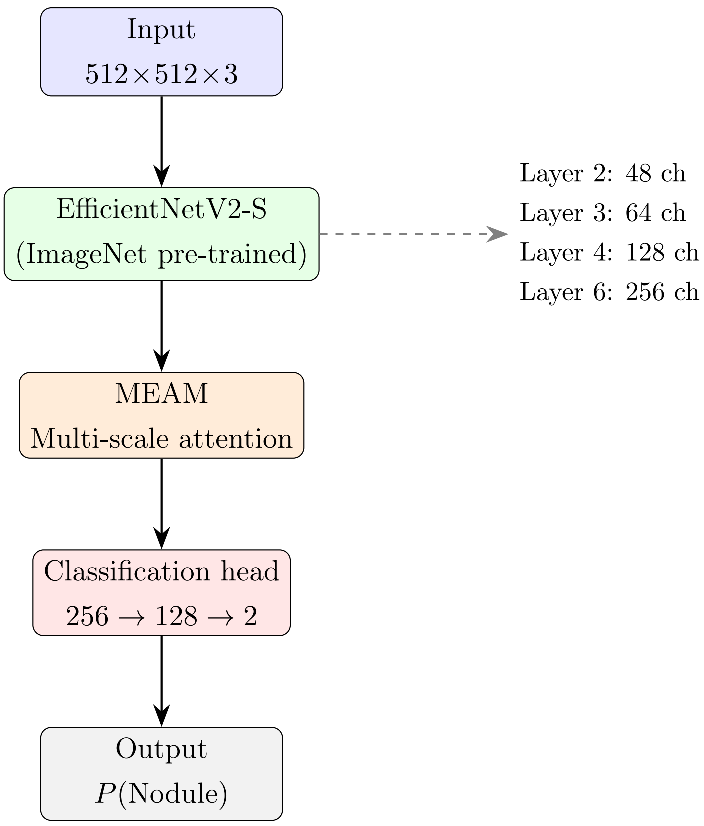

Figure 1: Global architecture of the `NoduleClassifier` classifier.

4.2 Backbone: EfficientNetV2-S

The backbone is an **EfficientNetV2-S** pre-trained on ImageNet-1K. Only the *features* part (without the original classification head) is kept. Feature maps are extracted at four intermediate levels:

Block index	**Channels**	**Relative resolution**
2	48	$\times\frac{1}{4}$
3	64	$\times\frac{1}{8}$
4	128	$\times\frac{1}{16}$
6	256	$\times\frac{1}{32}$

Table 3: Multi-scale extraction points for the `s` variant.

The `s` (Small) variant is used due to VRAM constraints.

4.3 MEAM module: multi-scale attention

The *Multi-Scale Embedding Attention Mechanism* fuses the four feature maps extracted from the backbone through a cross-scale attention mechanism.

4.3.1 Spatial alignment

All maps are first resized using bilinear interpolation to the resolution of the smallest one ($H_{\min} \times W_{\min}$, from block 6).

4.3.2 Q, K, V projections

For each scale $s \in \{1, \ldots, S\}$ (with $S = 4$), three 1×1 convolutions project the map $\mathbf{F}_s$ into query, key, and value spaces:

$$\mathbf{Q}_s = W_s^Q\,\mathbf{F}_s \quad \in \mathbb{R}^{B\times d_k\times HW} \tag{4}$$

$$\mathbf{K}_s = W_s^K\,\mathbf{F}_s \quad \in \mathbb{R}^{B\times d_k\times HW} \tag{5}$$

$$\mathbf{V}_s = W_s^V\,\mathbf{F}_s \quad \in \mathbb{R}^{B\times d_v\times HW} \tag{6}$$

with $d_k = d_v = 64$ by default.

4.3.3 Cross-scale attention

For each pair of scales (i, j), an attention score is computed:

$$A_{ij} = \operatorname{softmax}\left(\frac{\mathbf{Q}_i^\top \mathbf{K}_j}{\sqrt{d_k}}\right) \tag{7}$$

The output for scale i is obtained by summing contributions from all scales:

$$\hat{\mathbf{F}}_i = \sum_{j=1}^{S} \mathbf{V}_j \, A_{ij}^\top \tag{8}$$

4.3.4 Final fusion

The S outputs are concatenated along the channel dimension, then projected by a 1×1 convolution followed by *Batch Normalization* and a ReLU activation:

$$\mathbf{F}_{\text{fused}} = \operatorname{ReLU}\big(\operatorname{BN}\big(W^{\text{out}} \, [\hat{\mathbf{F}}_1 \,\|\, \cdots \,\|\, \hat{\mathbf{F}}_S]\big)\big) \in \mathbb{R}^{B \times 256 \times H_{\min} \times W_{\min}} \tag{9}$$

4.4 Classification head

The fused feature map goes through:

1. **Adaptive Average Pooling** $\rightarrow$ 256-dimensional vector,
2. linear layer $256 \rightarrow 128$ + ReLU,
3. **Dropout** ($p = 0{,}5$) for regularization,
4. linear layer $128 \rightarrow 2$ (two-class logits).

The probability of the `Nodule` class is obtained by applying the Softmax function to the logits: $P(\text{Nodule}) = \operatorname{softmax}(\mathbf{z})_1$.

5 Training strategy

5.1 Data split

The dataset is split into two subsets using a **stratified 80/20** split (scikit-learn `train_test_split`, `random_state=42`), ensuring that the nodule proportion is identical in both sets.

5.2 Class imbalance handling

The NIH dataset shows a strong imbalance between `Nodule` (minority) and `No Finding` (majority). Two mechanisms are combined:

- **WeightedRandomSampler**: oversamples the minority class. The number of samples per epoch is set to $2 \times n_{\text{nodules}}$ to balance classes.
- **Focal Loss** (Section 5.4): reduces the contribution of easy examples to the gradient.

5.3 Data augmentation

Applied only to the training set through the `Albumentations` library:

Transformation	**Probability**	**Parameters**
Horizontal flip	0,50	—
Shift / Scale / Rotation	0,55	±5% shift, ±5% scale, ±10
Brightness / Contrast	0,45	±10%
Elastic deformation	0,40	$\alpha = 120$, $\sigma = 6$, affine= 3,6

Table 4: Data augmentation pipeline.

5.4 Focal Loss

Focal Loss [?] modifies standard cross-entropy by weighting it with a factor $(1 - p_t)^\gamma$, which reduces the loss for confident predictions:

$$\mathcal{L}_{\mathrm{FL}}(p_t) \;=\; -\,(1 - p_t)^\gamma\;\log(p_t) \tag{10}$$

where p_t is the predicted probability for the correct class and $\gamma = 2{,}0$.

5.5 Optimization

Component	**Configuration**
Optimizer	AdamW (lr $= 10^{-4}$, *weight_ decay* $= 10^{-3}$)
Scheduler	Cosine Annealing ($T_{\max}$ = number of epochs)
Gradient clipping	Max norm = 1,0
Batch size	12
Resolution	512×512
Epochs	50
Seed	42

Table 5: Training hyperparameters.

5.6 Checkpoint criterion

The model is saved when **Average Precision** (AP) on the validation set reaches a new maximum. Two files are produced:

- `dernier_train.pth` — backbone weights (`model.backbone.state_dict()`), reusable,
- `best_dernier_train.pth` — full model (`model.state_dict()`).

```
--- Seuil optimal calculé (0.566) ---
Vrais Négatifs (TN): 11299 | Faux Positifs (FP): 779
Faux Négatifs (FN): 61    | Vrais Positifs (TP): 475
Sensibilité : 0.8862 | Spécificité : 0.9355
```

Figure 2: Model result

6 Evaluation

Model evaluation is performed through:

- the **ROC curve** and **AUC**,
- the **optimal threshold** using Youden's index ($\arg\max(\mathrm{TPR} - \mathrm{FPR})$),
- the **confusion matrix** at different decision thresholds (0,5; optimal threshold),
- **sensitivity** $= \frac{TP}{TP+FN}$ and **specificity** $= \frac{TN}{TN+FP}$.

7 Result

The model achieves 88,62% recall on the nodule class; this can be improved to 93,6% by lowering the threshold and slightly increasing the number of false positives (bringing it to 1068). It should nevertheless be noted that the model has relatively low precision on the nodule class. This could be compensated by a less severe class imbalance and more nodule images to improve generalization.

8 Architecture summary

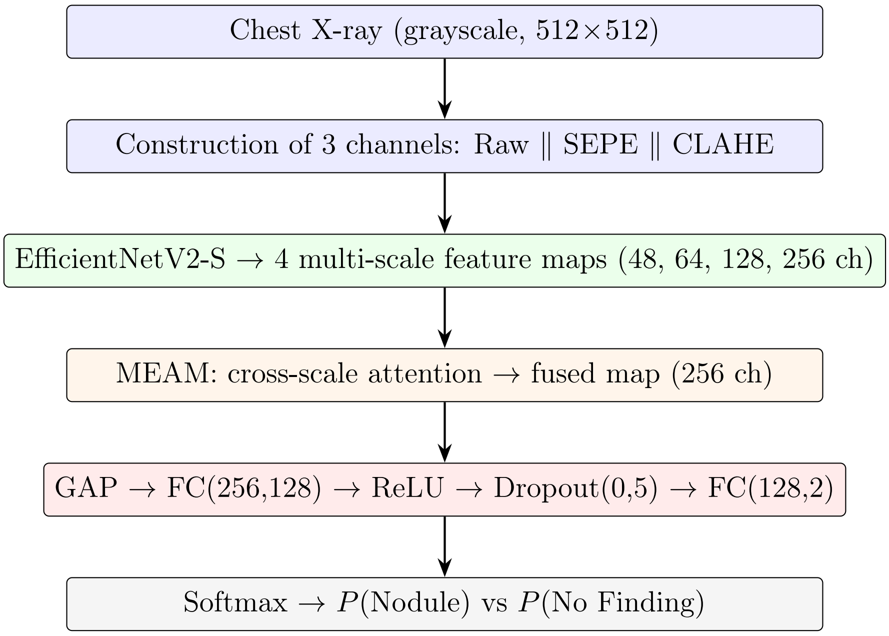

Figure 3: Complete pipeline, from raw image to prediction.

References

[1] M. Dhayalini, B. Revathi alias Ponmozhi, *Multi-phase deep learning framework with Multiscale Adaptive Swin Transformer and embedding attention for precision lung nodule detection and classification*, Scientific Reports, vol. 16, article 1652, 2025. doi:10.1038/s41598-025-31147-2

[2] S. Raza, R. Zia, I. A. Usmani, N. A. Almujally, N. Alasbali, M. Hanif, *Trans RCED-UNet3+: a hybrid CNN-transformer model for precise lung nodule segmentation*, Frontiers in Oncology, vol. 15, article 1654466, 2025. doi:10.3389/fonc.2025.1654466